Project Report – Non-Ideal Langmuir Systems

Dugan Haselow
Physics 525 | UW-Madison

28 April 2026

Abstract

A numerical model is used to simulate the dynamics of Langmuir waves and adjacent phenomena in non-ideal systems, allowing the observation of oscillatory behavior when conditions begin to deviate from the ideal regime. Computational work is conducted in a Python 3.14 environment with the help of general scientific libraries, including `numpy`, `scipy` and `matplotlib`. The model simulates a 1-dimensional particle-in-cell (PIC) system. Three main parameters are perturbed from the ideal Langmuir regime to observe outcomes. These include allowing mobile ions, non-negligible thermal velocity (exiting the cold-plasma limit), and non-quasineutrality. Most notably, we observe the square-root proportionality to number density in the cold-plasma formula for oscillation frequency and how universal constants re-manifest through simulation. We then linearize data supporting the warm-plasma dispersion relation and speculate on the causes of negative decay parameters possibly hinting at driven oscillations. Lastly, we observe how altering quasineutrality relates to the square-root proportionality of the cold-plasma frquency.

1 Purpose

With this project, I seek to answer questions surrounding the behavior of charged particles under different sets of conditions deviating from the ideal and approximate regimes in which we study Langmuir oscillations analytically. Using a numerical simulation of my own making, I hope to analyze what happens when minor imperfections are introduced to the ideal case and determine the point at which our analytical models for the phenomenon break down and give way to unique patterns.

2 Premise

In the ideal regime, Langmuir oscillations match the motion of a simple harmonic oscillator oscillating at the plasma frequency [1, p. 81]:

$$\omega_p = \left(\frac{n_0 e^2}{\varepsilon_0 m}\right)^{\frac{1}{2}} \quad \text{rad/sec}$$

However, this ideal scenario requires a load of assumptions [1, p. 78]:

- No magnetic field ($\vec{\mathbf{B}} = 0$)
- No thermal motions ($k_B T = 0$)
- Particle motion is confined to a single axis.
- Ions are considered fixed due to their high inertia.
- The plasma is infinite in extent.
- Quasineutrality inside the central zone ($n_e \approx n_i$)

The goal of this experiment is to go through the above list of constraints and to predict and observe how slight alterations breaking each of them can affect the overall behavior of the plasmic system. Some examples of these alterations include:

- Introduction of a weak magnetic field. [2, p. 245]
- Accounting for thermal motions.
- Allowing two- and three-dimensional particle motion.
- Considering motion of ions.
- Introducing a net charge imbalance.

This semester, I've focused entirely on a 1-dimensional particle-in-cell simulation. Since particles are confined to a cell with periodic boundaries, this satisfies the condition that the plasma is infinite in extent, as identical cells can be tiled across their boundaries. However, since this simulation is confined to one dimension, tweaks such as allowing two- and three-dimensional particle motion are unachievable. By proxy, this also disallows any possible influence from a magnetic field. This is due to the magnetic component of the Lorentz force $F_L = q(\vec{E} + \vec{v} \times \vec{B})$ being dictated by the cross product, making any magnetic forces orthogonal to the single spatial dimension of the system.

Therefore, the extent of this project is limited to observing through simulation how plasma frequency is affected with respect to non-negligible thermal motions, nonstationary ions, and a net charge imbalance in a 1-dimensional periodic universe.

3 Documentation and Workflow

3.1 Distribution Generator

The file `dist_gen.py` contains code necessary to generate the initial particle distributions used for this simulation. An (ideally) Maxwellian distribution function $f(\zeta)$ for some general

variable ζ is passed into the code, and using `numpy`, the distribution is numerically integrated and converted to probability space. A random number $\alpha \in [0, 1]$ is chosen, scaled to match the width of the probability space, then mapped back to the space of ζ within a window $[\zeta_{min}, \zeta_{max}]$ based on the bin α landed on in probability space. The random sampling and reverse mapping are repeated N times, and what results is a distribution of N points ζ_i for $i \in \{1, \cdots, N\}$, packed proportionally to $f(\zeta)$.

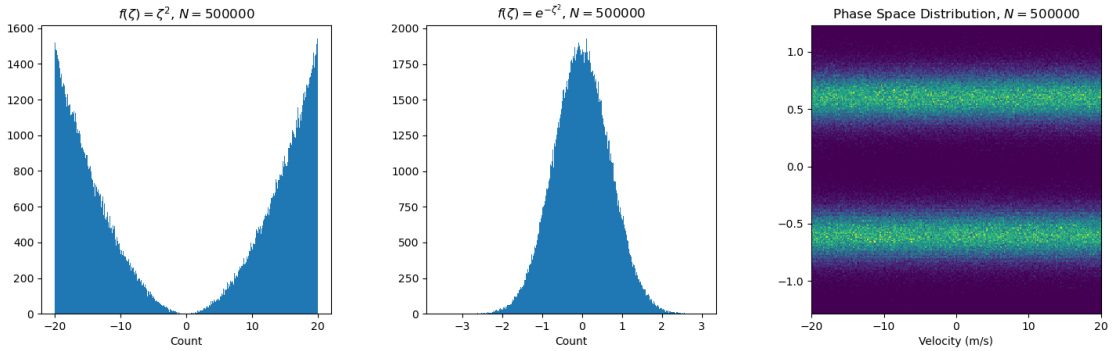


Figure 3.1.1. Example outputs from `dist_gen.py`.

Shown by the rightmost plot, this code is responsible for choosing the initial positions and velocities of particles during initialization. Note that $N = 500000$ is solely a stress test for the sampler code, not representative of particle counts used in experiments for this project.

For either species s, the functions defining the Maxwellian distributions are initialized with respect to thermal and drift velocities as follows:

$$f_{x,s}(x) = 1 \qquad f_{v,s}(v) = e^{-\left(\frac{v - v_{dr,s}}{v_{th,s}}\right)^2}$$

If `two_stream` is enabled, the velocity distribution is given by:

$$f_{v,s}(v) = e^{-\left(\frac{v - v_{dr,s}}{v_{th,s}}\right)^2} + e^{-\left(\frac{v + v_{dr,s}}{v_{th,s}}\right)^2}$$

Note that these distributions need not be normalized, as inside `dist_gen.py`, chosen randoms are automatically scaled to probability space.

3.2 Input Parameters

- `x_min, x_max`: Determine the position window for the simulation, and thus the cell length `x_width=x_max-x_min`.
- `v_min, v_max`: Determine the velocity window for the simulation, affecting the thermal velocity `v_th` in tandem with the parameters `v_th_ratio_e` and `v_th_ratio_i`.
- `steps_per_pd`: Determines the number of steps to take in the simulation for each period of the initial plasma frequency $\omega_{p,e}$, thus constraining the the time step `dt`.

- `n_frames`: Determines the total number of steps to take before ending the simulation.
- `window_res`: Determines the resolution (inverse to bin size) of 1D and 2D histograms in resultant plots.
- `f_cells_per_lam_D`: The number of field computation cells to assign for each Debye length λ_D.
- `max_field_res`: Determines the maximum number of field computation cells for ρ and electric field $\vec{E}$. `f_cells_per_lam_D` will be overridden if it exceeds this value.
- `fixed_ions`: If `True`, all ions in the simulation are initialized with zero velocity and remain unaffected by acting forces.
- `two_stream`: If `True`, the initial particle distributions will have two velocity peaks, spaced equally far from the origin by the drift velocities `v_dr_e` and `v_dr_i`.
- `rho_lim_scale, E_lim_scale`: Determine the ratio of the size of the vertical plot window to the initial maxima of the linear charge density and electric field, respectively.
- `ec_lim_margin`: Determines the multiplicative margin between the initial energy maxima and the top and bottom window edges of the logarithmic energy plot.
- `N_e, N_i`: The numbers of electrons and ions to generate upon initialization, respectively.
- `x_pert_amp_e, x_pert_amp_i`: Magnitude of sinusoidal number density perturbations along the x-axis.
- `x_pert_mode`: The integer-multiple spatial mode of the density perturbation.
- `v_th_ratio_e, v_th_ratio_i`: Determine the ratios of the thermal velocities ($\propto$ Gaussian peak width) to the width of the velocity window used for particle generation.
- `v_dr_ratio_e, v_dr_ratio_i`: Determine the ratios of the drift velocities `v_dr_e, v_dr_i` (Gaussian peak offset) to the width of the velocity window used for particle generation.
- `E_solver`: Chooses the solver to use in electric field computations. $0 \Rightarrow$ cumulative summation of ρ, $1 \Rightarrow$ Fast Fourier Transform (FFT) solve.
- `integration`: Determines the numerical integration scheme to use for particle motion. $0 \Rightarrow$ Velocity Verlet, $1 \Rightarrow$ Leapfrog (not used in this study due to poor energy conservation)

3.3 Plots

The simulation will output a figure containing six plots, evolving with time if the video export option is enabled.

The choice and formatting of these plots take after Rogerio Jorge's library `jaxincell`. The plots include:

- Two 2-dimensional histograms, denoting the phase space of electrons and ions, respectively. The bin width is determined inversely by the parameter `window_res`.
- A 1-dimensional histogram denoting the velocity distribution of electrons and ions, shown by the legend. The bin width is determined inversely by the parameter `window_res`.
- A plot showing the linear charge density with respect to position. Window height is determined by `rho_lim_scale`.
- A plot showing the electric field with respect to position. Window height is determined by `E_lim_scale`.
- A logarithmic energy plot showing the following with respect to time:
  - Total kinetic energy of electrons.
  - Total kinetic energy of ions. (Note: If `fixed_ions` is enabled, this will not show.)
  - Total potential of the electric field.
  - Total energy. (The sum of the above quantities)
  - Absolute error of the total energy with respect to its initial value.

  Window height is determined by `ec_lim_margin`. If animated, energy curves will be plotted live as the simulation evolves.

3.4 Initialization

3.4.1 Array initialization

The system starts with a dictionary for each particle type describing its attributes. For each particle added, the charge, mass, position and velocity of said particle are appended to lists containing the respective attribute for all particles. The contents of these lists are then written `numpy.array` instances, as these are much more time-efficient for large computations. Arrays are then calculated for the initial charge density and field based on the distribution of particles, allowing the computation of initial particle accelerations.

3.4.2 Diagnostics

After plots are initialized, diagnostics are printed to the command line. These include a handful of input parameters, as well as some which are computed upon initialization:

- Time step: Δt (a.k.a. `dt`) $= \frac{2\pi}{\texttt{steps\_per\_pd}\cdot\omega_{p,e}}$.
- Electron plasma frequency: $\omega_{p,e} = \sqrt{\frac{n_0 e^2}{m_e \varepsilon_0}}$. **Note**: n_0 is calculated as a ratio of the total number of electrons to the length of the simulation region, as generated distributions are uniform with respect to position.
- Electron plasma period: $\tau_{p,e} = \frac{2\pi}{\omega_{p,e}}$

- Electron/ion thermal velocities: $v_{th,s} = \texttt{v\_th\_s\_ratio} \cdot \texttt{v\_width}$, where s denotes the species $\in \{e, i\}$.
- Electron/ion temperatures: $T_s = \frac{m_s v_{th,s}^2}{2k_B}$, where s denotes the species $\in \{e, i\}$.
- Electron Debye length: $\lambda_{D,e} = \sqrt{\frac{\varepsilon_0 k_B T_e}{n_{0,e} e^2}} = \frac{v_{th,e}}{\sqrt{2} \cdot \omega_{p,e}}$

3.5 Simulation

3.5.1 Calling the simulation

The function called to start and run the simulation is `pic_1d(<inp_params>, <export_anim_filename>, <show_plot>, <verbose>)`, located inside `picell_1d.py`.

The arguments of the function act as follows:

- `inp_params`: A dictionary containing key strings specifying the names of input parameters (see Section 3.2), accompanied by values specifying the set values. If not specified, the system will assume default values corresponding to a generalized two-stream instability simulation.

  Example:

```
params: dict = {
    "x_min": -2e-11,
    "x_max": 2e-11,
    "v_min": -0.025,
    "v_max": 0.025,
    "steps_per_pd": 20,
    # ...
    "N_i": 20000,
    "v_th_ratio_e": 1/600,
    "v_th_ratio_i": 1/600,
    "v_dr_ratio_e": 1/8,
    "v_dr_ratio_i": 1/8
}
pic_1d(params)
```

- `export_anim_filename`: A local or system file path specifying the name and format of the file you want to export the animation to. This argument should support all file types which are supported by `matplotlib.animation`. If not specified, no animation will be created or exported.

  Example:

```
pic_1d(params, r"outputs\pic_1d_output.mp4")
```

Note: Rendering and exporting an animation takes significantly longer than running the simulation without exporting.

- `show_plot`: Tells the simulation whether to show the final plotted frame upon finishing.
- `verbose`: Tells the simulation whether to print out diagnostics during runtime.

3.5.2 Main Processes

After the system is initialized, `export_anim_filename` is evaluated. If the string contains a valid file path, the main loop will be run by `matplotlib.animation`, which runs significantly slower due to rendering overhead. If no string is specified, the loop will be carried out by a simple Python `for` loop, running significantly faster.

For each time step, particles evolve in time using the Velocity Verlet integration method, given as follows:

$$x(t + \Delta t) = x(t) + v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2$$

$$v(t + \Delta t) = v(t) + \frac{a(t) + a(t + \Delta t)}{2}\Delta t$$

Note: $a(t) = a(x(t)) \Rightarrow a(t + \Delta t) = a(x(t + \Delta t))$.

Between the computation of the new position and velocity, the charge density and electric field must be recomputed to find $a(t + \Delta t)$. This is done by adding the elementary charges of each particle to the corresponding cell of an array containing the charge, then normalizing to the length of the simulation region. The electric field is then calculated using one of two methods:

- A cumulative sum over the array of charge density:

$$\nabla \cdot E = \frac{\mathrm{d}E}{\mathrm{d}x} = \frac{\rho}{\varepsilon_0} \Rightarrow E(x) = \frac{1}{\varepsilon_0}\int_{x_0}^{x} \rho(x')\mathrm{d}x' \Rightarrow E_n = E(x_0 + n\Delta x) \approx \frac{1}{\varepsilon_0}\sum_{i=0}^{n} \rho_i$$

- A Fast Fourier Transform (FFT) solve:

$$\frac{\mathrm{d}}{\mathrm{d}x} \xrightarrow{\text{FFT}} ik \Rightarrow \frac{\mathrm{d}E}{\mathrm{d}x} = \frac{\rho(x)}{\varepsilon_0} \xrightarrow{\text{FFT}} ik\bar{E}(k) = \frac{\bar{\rho}(k)}{\varepsilon_0}$$

$$\rho(x) \xrightarrow{\text{FFT}} \bar{\rho}(k) \xrightarrow{1/ik\varepsilon_0} \bar{E}(k) \xrightarrow{\text{iFFT}} E(x)$$

For all intents and purposes, we will be using the FFT solving method, facilitated by the use of `numpy`.

From this point, the new acceleration on all particles i is calculated using the Lorentz force with $B = 0$ (1D constraint):

$$F_{L,i} = m_i a_i \Rightarrow a_i(x_i) = \frac{F_{L,i}}{m_i} = \frac{q_i E(x_i)}{m_i}$$

After the velocity computation, the new acceleration is stored as the current acceleration. This pipeline is repeated until the total number of iterations hits `n_frames`.

Once the simulation finishes, the last frame to be processed is optionally shown. The time-dependent electric field $E(t)$ is analyzed to find the observed frequency ω_{obs}, as well as other useful values such as the decay coefficient γ. These are then returned by the function and optionally printed as end diagnostics for analysis.

4 Experiments

To test how the frequencies of plasma oscillations are affected by different factors, we pass in to the simulation variances in number density, thermal velocity, ion mobility, and charge balance. These variances will deviate from a set of parameters representing the ideal case for plasma oscillations.

Frequencies are measured using the function `curve_fit` from the library `scipy.optimize`, falling back on a Fast Fourier Transform (FFT) method if descent is unstable. The following function is used to determine the curve fit parameters:

$$\texttt{sinusoid}(t) = Ae^{-\gamma t}\sin(\omega t + \phi) + C$$

Tests will mainly focus on the observed oscillation frequency ω and the decay rate γ, corresponding to a complex frequency $\omega - i\gamma$.

4.1 Ideal Regime

To emulate the ideal regime, we choose the following set of initial parameters:

```
inp_params: dict = {
    "x_min":   -0.00005,
    "x_max":    0.00005,
    "v_min":   -10,
    "v_max":    10,
    "steps_per_pd":   75,
    "n_frames":    500,
    "window_res":   250,
    "f_cells_per_lam_D": 2000,
```

```
    "max_field_res":  5000,
    "fixed_ions":   True,
    "two_stream":   False,
    "rho_lim_scale":  15,
    "E_lim_scale":   10,
    "ec_lim_margin":  10,
    "N_e":     50000,
    "N_i":     50000,
    "x_pert_amp_e":   0.2,
    "x_pert_amp_i":   0,
    "x_pert_mode":       1,
    "v_th_ratio_e":   1/5000,
    "v_th_ratio_i":   1/5000,
    "v_dr_ratio_e":   0,
    "v_dr_ratio_i":   0,
    "E_solver":    1,
    "integration":   0
}
```

Said parameters may be arbitrary on some axes; however, this sets a controlled precedent for the variations we will introduce.

4.2 Number Density (n_0)

The basic plasma oscillation frequency is defined as

$$\omega_{p,e} = \sqrt{\frac{n_0 e^2}{m_e \varepsilon_0}} = \sqrt{\frac{e^2}{m_e \varepsilon_0}} \cdot \sqrt{n_0} \approx 56.415 \cdot \sqrt{n_0}.$$

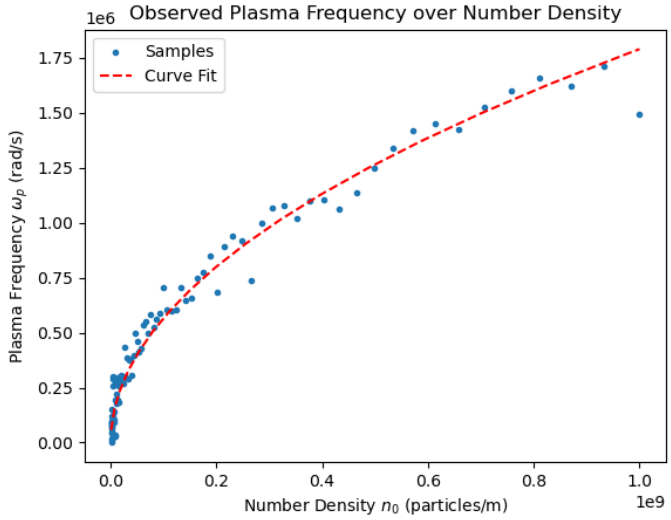
To create a sweep of the number densities n_0, a logarithmically-distributed set of 100 different particle counts N_0 ranging from 100 to 100,000 was created and then normalized to the width of the simulation box to determine the number density of particles.

For each of 100 samples, the particle number is initialized into the simulation parameters and the simulation is run. The frequency determined by `curve_fit` is then recorded.

Once the samples are collected, a new `curve_fit` is then created for the following function:

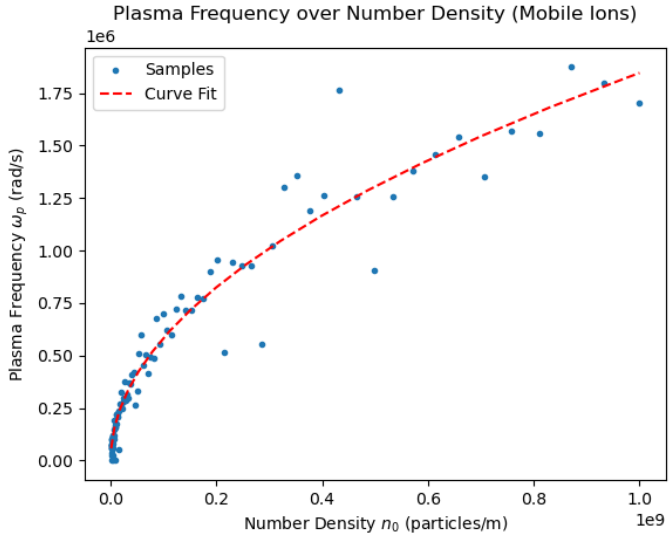
$$\omega(n_0) = A\sqrt{n_0}$$

Given the first relation, we should expect to see that the parameter A will hover around the value of $\sqrt{\frac{e^2}{m_e \varepsilon_0}} \approx 56.415$. The plot below shows the optimized $\omega(n_0)$ plotted against the scatter of samples.



For this sample, `curve_fit` optimization yielded $A = 56.59 \pm 0.61 \frac{\text{rad}\cdot\text{m}^{1/2}}{\text{s}}$, matching the expected scientific value within `scipy`'s confidence margin, and with an error of 0.3%.

Running the same sweep while allowing ion motion at the same small thermal scale as the electrons (`fixed_ions = False`), we see the following:



As expected, allowing motion of ions introduces more noise to our frequency samples. This time, the fitted curve yielded $A = 58.36 \pm 0.93 \frac{\text{rad}\cdot\text{m}^{1/2}}{\text{s}}$, sitting close to but outside of the confidence interval, and giving an error of 3.5%.

4.3 Thermal Velocity (v_{th})

To relate the thermal velocity of a plasma system to its oscillation frequency, one may look toward the the warm-plasma dispersion relation [1, p. 228]:

$$\omega_{obs}^2 = \omega_p^2 + \frac{3KT}{m}k^2 = \omega_p^2 + 3k^2 v_{th}^2,$$

where $k = \frac{2\pi n}{L}$ is the perturbation wavenumber and $\omega_p = \sqrt{\frac{n_0 e^2}{m_e \varepsilon_0}}$ is the cold plasma frequency.

To see any noticeable results, we need $\omega_p^2 \sim 3k^2 v_{th}^2$. From here, we can use the identities $\omega_p^2 = \frac{n_0 e^2}{m_e \varepsilon_0}$, $k = \frac{2\pi n}{L}$ and $n_0 = N/L$:

$$\frac{v_{th}^2}{\omega_p^2} \sim \frac{1}{3k^2} \Rightarrow v_{th}^2 \frac{L n_e \varepsilon_0}{N e^2} \sim \frac{L^2}{6\pi^2 n^2} \Rightarrow v_{th}^2 \sim \frac{e^2}{m_e \varepsilon_0} \frac{NL}{6\pi^2 n^2}$$

If we set $N = 20000$, $L = 0.0001$ and $n = 1$ as in our base parameters, we evaluate the RHS to find that v_{th} must be roughly on the order of 10 to have a substantial effect on the observed frequency.

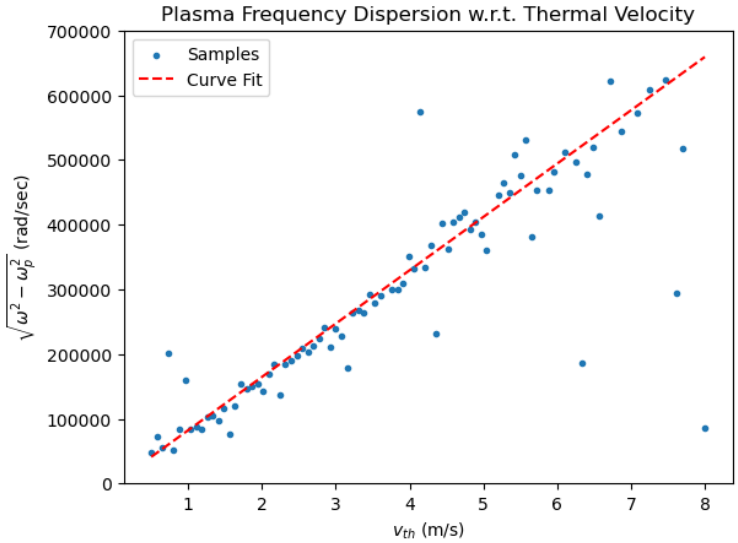
Simulations are then run, logging frequencies as the system sweeps over 100 values of v_{th}. We then linearize our data:

$$\omega^2 = \omega_p^2 + 3k^3 v_{th}^2 \Rightarrow \sqrt{\omega^2 - \omega_p^2} = \sqrt{3} \cdot k v_{th} = \sqrt{3} \cdot \frac{2\pi n}{L} v_{th}$$

Substituting $L = 0.0001$ and $n = 1$, we should expect that

$$\sqrt{\omega^2 - \omega_p^2} \approx 1.09 \cdot 10^5 \cdot v_{th}.$$

Plotting the augmented frequency data $\sqrt{\omega^2 - \omega_p^2}$ over v_{th}, we create a simple linear fit $\sqrt{\omega^2 - \omega_p^2} = A \cdot v_{th}$:



The curve fit yielded the parameter $A = 82400 \pm 1500 \, \frac{\text{rad}}{\text{m}}$. This is roughly 24% lower than the expected value for $n = 1$. This is most likely a simple fact of particle versus fluid simulations, as the warm-plasma dispersion relation is derived from the fluid approximation. Nevertheless, we've shown that a clear proportionality arises from this dataset.

Another factor to consider with respect to thermal velocity is the system's susceptibility to Landau damping. From Chen [1, p. 229], we have:

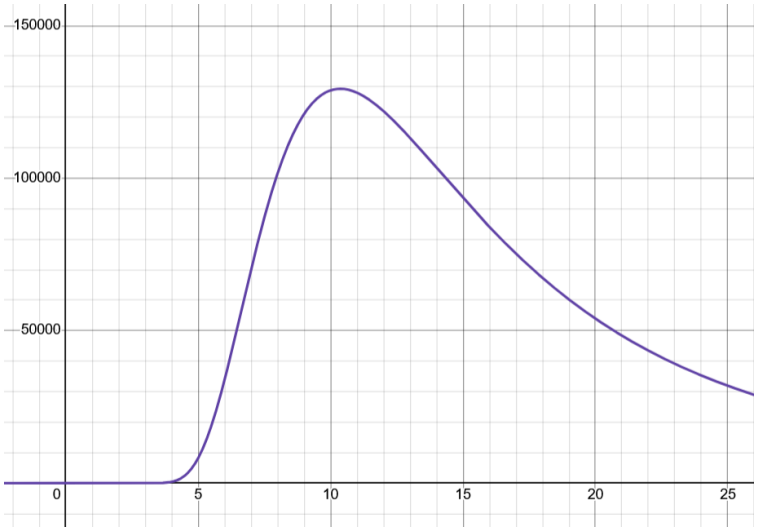
$$\text{Im}(\omega) = -\frac{\pi}{2}\frac{\omega_p^3}{k^2}\frac{2\omega_p}{k\sqrt{\pi}}\frac{1}{v_{th}^3}\exp\left(-\frac{\omega^2}{k^2 v_{th}^2}\right) = -\frac{\sqrt{\pi}\omega_p^4}{k^3 v_{th}^3}\exp\left(-\frac{\omega^2}{k^2 v_{th}^2}\right)$$

$$\text{Im}(\omega) = -\sqrt{\pi}\omega_p\left(\frac{\omega_p}{k v_{th}}\right)^3\exp\left(-\frac{\omega_p^2}{k^2 v_{th}^2}\right)\exp\left(-\frac{3}{2}\right)$$

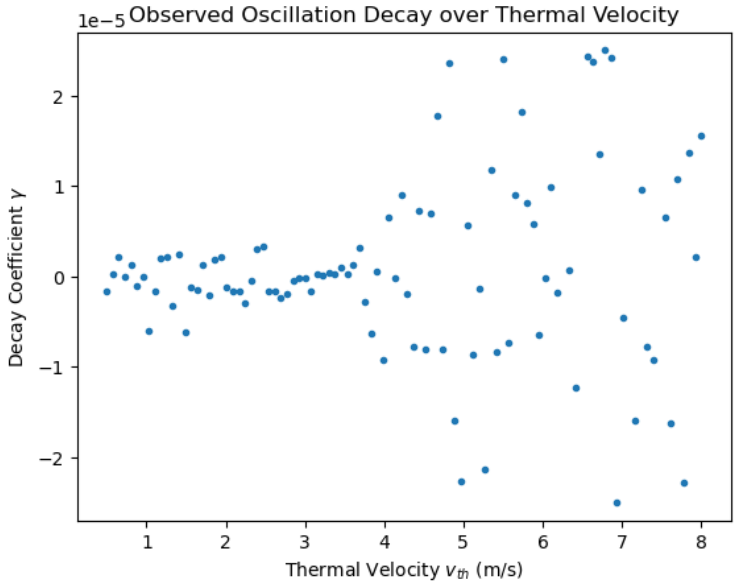
Recall that the curve fit used to estimate rates of decay is proportional to $e^{-\gamma t}$. Therefore, we should expect that

$$\gamma = -\text{Im}(\omega) \approx \sqrt{\pi}\omega_p\left(\frac{\omega_p}{k v_{th}}\right)^3\exp\left(-\frac{\omega_p^2}{k^2 v_{th}^2}\right)\exp\left(-\frac{3}{2}\right).$$

This curve takes the following shape:



However, when plotting the data, we see that past the threshold $v_{th} \approx 4$, amplified noise begins to disperse points for γ nearly symmetrically in both the positive (decay) and negative (growth) directions. If this plot has basis in reality, the presence of points $\gamma < 0$ is indicative of inverse Landau damping, implying driven oscillations in the plasma system.



The envelope containing this noise may be proportional to the above relation, though this is entirely speculation.

4.4 Charge Imbalance ($N_e \neq N_i$)

This experiment focuses varying the ratio of electrons to ions, thereby affecting the net charge of the system and perturbing the system from quasineutrality.

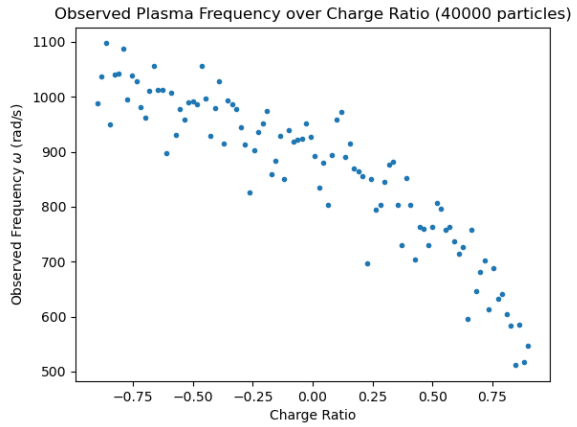
We will operate in the metric of a "charge density ratio", defined as follows, where R_q becomes the independent variable:

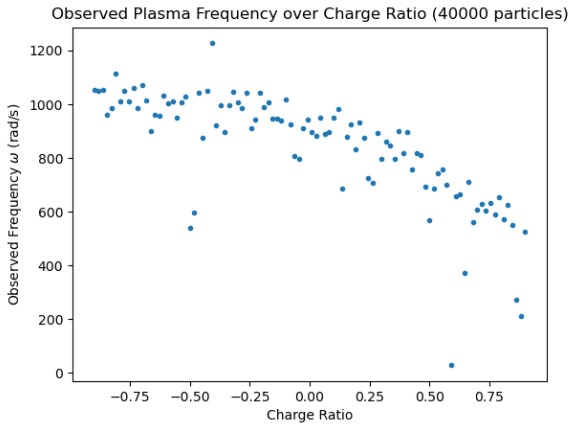
$$R_q = \frac{N_i - N_e}{N_i + N_e}$$

Thus, when the system is comprised entirely of ions, $R_q = 1$. Similarly, $R_q = -1$ when the system contains only electrons, and $R_q = 0$ when the system is neutral.

The variable sweep takes 100 values of R_q between -0.9 and 0.9, running simulations with $N_i + N_e = 40000$ total particles, thus ranging the particle concentrations from 90% electrons to 90% ions.

Two sweeps were conducted, pictured below – one with fixed ions (left), the other with mobile ions (right):





We see once again that allowing ion motion introduces noise into the system, evidenced by the increased presence of outlying points. This aside, because ions are at least nearly immobile in both cases, we see that the observed frequency decreases in a manner $\propto \sqrt{1 - R_q} = \sqrt{\frac{2N_e}{N_i+N_e}}$. Because the box width L stays the same, this dependence on N_e directly affects the number density n_0 of mobile particles (i.e. electrons), aligning as expected with the proportionality of the cold plasma oscillation frequency $\omega_p = \sqrt{\frac{n_0 e^2}{m_e \varepsilon_0}}$.

5 Endnotes

5.1 Conclusions

- $\sqrt{\frac{e^2}{m_e \varepsilon_0}} = 56.59 \pm 0.61$ (Fixed Ions)
- $\sqrt{\frac{e^2}{m_e \varepsilon_0}} = 58.36 \pm 0.93$ (Mobile Ions)
- $\sqrt{3} \cdot \frac{2\pi n}{L} = 82400 \pm 1500$ (-24% from expected value; underlying cause unknown)
- $\omega(R_q) \propto \sqrt{1 - R_q}$

5.2 Tools Used

Simulations were written and computed in a Python 3.14 environment with the aid of libraries including `scipy`, `numpy`, and `matplotlib`. Jupyter notebooks were used as a visual interface for creating and exporting plots, while the main internals of the simulation are stored inside a regular Python script file. Changes to the code were made using Microsoft Visual Studio Code and committed to a dedicated code repository on GitHub. AI was utilized as a light aid in debugging, and was not used to generate new code. All simulation code was typed and edited by hand.

5.3 Code

All code used to write this report is hosted on the following GitHub repository:

https://github.com/dchaselow/sp26-plasma-research-project

This includes:

- The main simulation script (`.py`)
- The distribution generator (`.py`)
- Plots from Sweeping Tests and code required to reproduce them (`.ipynb`)
- Old / legacy scripts
- Demonstration Video Graphics

6 Sources

[1] Francis Chen. *Introduction to Plasma Physics and Controlled Fusion*. Cham Springer International Publishing, 2016.

[2] C Sulem and P L Sulem. *The nonlinear Schrödinger equation : self-focusing and wave collapse*. Springer, 1999, pp. 245–262.